SAFEX SOLUTIONS / WEEK 4 / GROUP 57

Which language model should answer our customers first?

A vendor evaluation for a courier operation deciding which model reads its inbound customer support first. Three providers, one prompt, thirty hand-labelled cases, one weighted recommendation.

Progress report / Fatima Naseer / Institute of Space Technology

9 August 2026

Field	Detail
Candidate	Fatima Naseer, Institute of Space Technology, Islamabad
Group	57, Group Leader Salman Aslam
Week	4, AI and ML department, client-ready sprint
Project	Which language model should answer our customers first
Use case	Inbound courier support triage and first reply
Vendors tested	OpenAI, Anthropic, Google, matched on small and fast tier
Status	Harness, eval set, dashboard and memo complete. Live run pending.

READ THIS FIRST

The dashboard shipped with this report is currently rendering synthetic data, generated so the layout could be reviewed before spending API credit. It carries a red band across the top saying so. The benchmark has to be run against live keys before any figure in it is quoted to anyone. Nothing in this progress report claims a measured result.

01

Objective and scope

Compare three language model providers on one narrow job and recommend one, with enough working shown that somebody who disagrees can check the reasoning rather than argue about taste.

The job is first-line triage of inbound customer support for a courier business. One message arrives on WhatsApp or web chat, in English, Roman Urdu, or the mix people actually type. The model returns a JSON object carrying an intent label, three extracted entities, a routing decision, and a customer-facing reply. The JSON goes to the ticketing system and the reply goes to the customer.

This continues the courier business case from Week 3 rather than starting a new one. Week 3 established the operation; Week 4 answers a build decision inside it.

Why this scope and not a general comparison

A general capability ranking would have been faster to produce and worth nothing to a decision maker. The question in front of an operations lead is not which model is cleverest. It is which one labels forty thousand messages a month correctly, cheaply, quickly, and without telling a customer something it invented.

02

What was built

Component	Path	What it is
Evaluation set	eval/eval_set.json	Thirty cases labelled by hand before any model ran. Eight intents plus an out-of-scope class. Nine Roman Urdu, six mixed, fifteen English. Six deliberately adversarial.
Shared prompt	eval/system_prompt.txt	One file, sent byte for byte to all three vendors. Its SHA is printed in the report header so nobody has to take that on trust.
Vendor clients	bench/providers.py	Raw HTTP per vendor rather than SDKs, plus an optional path through a local LiteLLM gateway.
Scoring	bench/scoring.py	Six deterministic scorers, Wilson confidence intervals, percentile helpers. No model grades another model.
Runner	bench/run_benchmark.py	Concurrency, backoff on 429 only, per-vendor runs that merge into one results file, prompt-hash guard against mixing two prompts in one table.
Aggregation	report/aggregate.py	Every figure in the report is computed here and nowhere else, so there is one place a number can be wrong.
Dashboard	report/dashboard.html	Comparison table, accuracy breakdown, latency percentiles, cost against the human baseline, sensitivity analysis, side-by-side sample outputs, failure log.
Memo	docs/recommendation_memo.md	Generated from the same summary object as the dashboard, so the two cannot drift apart.
Methodology	docs/methodology.md	Every measure defined, including the six-line integration rubric that is scored by hand.

Component	Path	What it is
Workbooks	trackers/	Outreach tracker with dropdowns and a rules sheet, plus a live-formula scorecard so the weights can be argued with directly.
Outreach	docs/outreach.md	Three target organisations with the reasoning, and one message each.
Video script	docs/video_script.md	Seven-minute walkthrough, timed, with what to show on screen.

03

Method, in short

Vendors were matched on tier rather than on brand. All three models are each vendor's small and fast tier, the class you would actually deploy at this volume. Comparing a frontier model from one vendor against a budget model from another produces a number that looks like a finding and means nothing.

Axis	Weight	How it is scored
Accuracy	40%	Composite of six components: schema validity 20, intent 25, entity F1 20, routing action 15, grounding 10, policy facts 10. A reply that does not parse scores zero on all six.
Cost	25%	Token counts from each vendor's own usage field, priced at rates in config/pricing.json with the date they were checked. Cheapest scores 100, the rest scale by ratio.
Latency	20%	Scored on p95, not mean, because on WhatsApp the customers who leave are the ones in the tail. Full marks at or below the target, zero at or above the failure threshold.
Integration	15%	Six-line rubric scored by hand after writing the client code: SDK quality, structured output, docs, error semantics, rate limit headroom, data residency.

Fairness controls

- One prompt file, read once, hashed, sent identically to all three.
- Temperature zero everywhere, three repeats per case, so run-to-run variance shows up as a stability number instead of disappearing into an average.
- Raw HTTP rather than vendor SDKs, because a vendor whose SDK retries silently would post better latency for a reason that has nothing to do with the model.
- Same machine, same connection, same session for all three.
- The weighted score is recomputed under five different weight profiles. If the winner changes, the memo names the assumption the decision rests on instead of hiding it.

04

Status

Deliverable	State	What remains
Comparison table	Built, unpopulated	Run the benchmark against live keys

Deliverable	State	What remains
Sample outputs from each provider	Built, unpopulated	Same run fills this
Recommendation memo	Generated, awaiting real figures	Regenerates automatically after the run
Integration rubric	Rubric written, unscored	Score by hand after writing client code against each API
Outreach messages	Written	Identify the right contact at each company, then send
Outreach tracker	Built	One row per message on the day it goes out
Walkthrough video	Scripted	Record after the dashboard holds real numbers
Presentation slides	Built	Update the three figure slides after the run

WHY THE NUMBERS ARE NOT HERE YET

A full run is thirty cases times three repeats times three vendors, which is two hundred and seventy requests. Free tiers rate-limit partway through that, which leaves a half-finished results file. The runner was built to handle it by running one vendor at a time and merging into the same file, but the run itself needs live keys on all three accounts. Publishing invented figures to fill the gap would have made the report look finished and made it worthless.

05

Daily work log

Fill one row per working day, with actual hours. The first row shows the expected format and should be deleted before submission.

Date	Day	Hours	What I worked on	What got in the way
Example, delete this row	Mon	3.5	Drafted the eval set, wrote twelve of the thirty cases	Kept getting the Roman Urdu register wrong, went back to real WhatsApp screenshots
	Mon			
	Tue			
	Wed			
	Thu			
	Fri			
	Sat			

06

Problems encountered and how they were solved

Vendor SDKs are not comparable

Problem. Some SDKs retry internally and some do not. A vendor that retries silently posts better latency for a reason that has nothing to do with its model.

Solution. Wrote the HTTP calls by hand, one small function per vendor, with identical timeout and retry policy. Writing them also turned into the evidence for the integration score.

Models wrapped valid JSON in a code fence

Problem. Counting a fenced reply as invalid would have overstated the failure rate. Counting it as clean would have hidden a real integration cost, since production code has to strip it.

Solution. Split into two measures: schema validity, and a separate clean-JSON rate that records how often the raw text needed unwrapping. Both appear in the comparison table.

Entity scoring rewarded returning nothing

Problem. Most cases have at least one genuinely empty slot. A first version of the F1 counted those as correct, so a model that answered null to everything scored well.

Solution. Slots empty in both the gold and the prediction are now excluded from the F1 entirely, so nulls earn nothing.

English keyword matching failed Roman Urdu replies

Problem. A correct Urdu reply was scoring zero on policy coverage because the required fact was checked as an English string.

Solution. Each required fact became a set of acceptable surface forms across both languages. A reply gets the mark for saying the thing, in whichever language the customer used.

Free-tier rate limits on a 270-request run

Problem. Hitting a 429 partway through leaves a half-written results file and no record of which half is real.

Solution. Backoff on 429 only, other errors recorded rather than retried, and per-vendor runs that merge into one file as long as the prompt hash matches. A smoke-test mode runs nine requests first.

Thirty cases cannot separate close vendors

Problem. A bare percentage at this sample size implies precision the evidence does not support.

Solution. Added Wilson confidence intervals to the intent accuracy row and stated a ten-point tie threshold in the memo, the methodology, and the dashboard itself.

The weights are a judgement, not a measurement

Problem. Anyone can object that the weights were chosen to produce a preferred answer.

Solution. The report recomputes the ranking under five weight profiles including cost-first and speed-first, and says in plain words whether the winner survives all five.

Risk of synthetic preview data being mistaken for a result

Problem. A demo dataset built to check the layout is exactly the kind of thing that gets submitted by accident at 2am.

Solution. The results file carries a provenance field, the dashboard renders a red band whenever it reads DEMO, the build script prints a warning, and the demo file is gitignored.

07

Client outreach

Three e-commerce logistics companies in Pakistan, all tech-forward enough to already be thinking about support automation, and all small enough that a comparison like this is useful rather than something they have already paid a consultancy for.

Organisation	Why they fit	Channel
PostEx	Courier plus payments for e-commerce sellers, so support carries both delivery questions and money questions. The cash-on-delivery cases in the eval set are exactly their message mix.	LinkedIn
Rider	Karachi last-mile built around online shippers, founded 2019, API-first. Volume high enough for per-contact support cost to be a real line item.	LinkedIn
Unity Retail	Sells fulfilment software to merchants rather than delivering parcels, so their support conversations are with sellers rather than consumers. Same triage problem, different message mix.	LinkedIn

Rules held to

- One message per organisation, written and sent by hand. No automation, no list send.
- The message states plainly that I am an intern and that I am not selling anything.
- No claim to have studied their operation, because I have not. The offer is to show the method and ask where the test cases are wrong.
- A not-interested reply closes the row on the day it arrives, with no follow-up later.
- No reply after ten days is logged as no reply and the row closes.
- If a conversation ever reached paid work it would go through SafeX Solutions, and I would say so during the call rather than after it.

08

Next week

Priority	Task	Done when
1	Run the full benchmark against live keys on all three vendors, one vendor at a time if rate limits bite	results/results.json holds 270 records and the dashboard band reads Measured
2	Score the integration rubric by hand after writing client code against each API and deliberately triggering a few errors	All eighteen rubric cells filled in config.yaml
3	Rebuild the dashboard and memo, update the three figure slides	Red band gone, memo carries real numbers
4	Record the seven-minute walkthrough	1080p, face visible on the open and close

Priority	Task	Done when
5	Find the right contact at each of the three companies and send the messages	Three rows in the tracker with dates
6	Expand the eval set toward a hundred cases and get a second person to label a sample independently	Inter-annotator agreement measured and reported
7	Add prompt injection cases, since a customer message is untrusted input and nothing in this round tests it	At least ten injection cases with gold behaviour defined
8	Rerun latency from the production region rather than a laptop	p95 figures the ops team would accept in an SLA conversation

Items 6 and 7 are the ones that would change the quality of the answer rather than its presentation. Thirty cases is enough to rule a vendor out and not enough to sign a contract, and untested prompt injection on a channel where the input is written by strangers is the largest open risk in the work.